论文编号：202208064722（学号）

本科毕业论文（设计）

基于 RAG 与工作流的中原工学院招生咨询系统

Admission Consulting System of Zhongyuan University of Technology Based on RAG and Workflow

学院名称： 人工智能学院

专　　业： 人工智能

班　　级： 人工智能 221

学　　号： 202208064722

学生姓名： 曾令洁

指导教师： 车战斌

2026　年　5　月

诚信承诺书

本人郑重声明：本人所呈交的毕业论文（设计），是在导师的指导下独立进行研究所取得的成果。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或在网上发表的论文。

论文作者签名：_____ 日期：_____

摘　要

高校招生咨询的难点，不在于“问题多”，而在于“问题杂、来源散、口径不能错”。同一轮咨询里，考生可能先问招生章程，再追问某个专业的学费、选考科目和学院网址，接着还要继续问报到流程或者资助政策。传统 FAQ 系统覆盖面有限，人工热线又受时间和人力约束；直接使用大语言模型虽然交互自然，但在招生这类高谨慎场景下，若回答没有证据约束，就很容易出现事实幻觉或年份混淆。围绕这些实际问题，本文以中原工学院招生资料为基础，设计并实现了一套基于检索增强生成（Retrieval-Augmented Generation，RAG）与工作流编排的招生咨询系统。

系统采用前后端分离架构。前端基于 Vue 3、TypeScript 与 Vite 构建多会话咨询界面，支持普通对话、RAG 问答与专家模式切换，并通过 SSE 方式流式展示回答过程。后端以 FastAPI 作为统一入口，在网关层组织查询改写、结构化路由、混合检索、引用校验、答案生成、记忆接入、技能调用和故障降级。知识层同时维护非结构化文档与结构化招生数据：一方面对招生章程、报考指南、学院介绍等资料进行解析、切块和索引，构建 FAISS 向量检索与 BM25 稀疏检索链路；另一方面将专业目录、分数线、政策附表等字段数据存入 MySQL，以支撑费用、选科、专业代码和位次类精确查询。

在核心方法上，本文没有采用单一路径检索，而是设计了“稠密检索 + 稀疏检索 + 精确匹配 + RRF 融合 + 重排 + 质量闸门”的混合检索链路，并进一步结合工作流编排，把复杂咨询拆分为可追踪的执行步骤。针对招生业务对可靠性的要求，系统还实现了引用校验、冲突裁决、输入输出安全审查、健康检查、隔离执行器和多级降级链，尽量避免在证据不足时给出武断回答。

基于项目现有脚本与报告，本文从问答正确性、检索效果、消融结果、性能表现和容错能力五个方面进行了验证。当前招生 RAG 专项评测覆盖 70 个样本，工具路由准确率达到 100%，整体通过率为 55.71%，Recall@5、MRR@5 和 nDCG@5 分别为 0.6976、0.9857 和 0.8078；结果表明系统在“路由到正确链路”和“召回相关证

据”方面表现较好，但学院介绍和报考指南类问题的答案忠实性仍有明显改进空间。问答质量抽样评测中，关键词覆盖率与引用命中率均为 100%，幻觉率为 0。网关故障回放测试显示，5 组核心样例均能返回有效结果，P95 延迟为 13.98 ms；并发压测则暴露出主链路时延偏高的问题，在并发 5 时 P95 首字节时间已达到 48.77 s，说明系统当前更适合课程原型与受控演示环境，尚不适合直接面向高并发生产场景。

总体来看，本文工作完成了一套面向真实高校招生业务的智能咨询原型，验证了“结构化查询 + 混合检索 + 工作流编排 + 可靠性控制”这一技术路线的可行性。系统仍存在检索重排策略需继续调优、记忆持久化尚未落地、部署架构有待收敛等问题，但其在回答准确性、证据可追溯性和工程可维护性方面已具备较为清晰的实现基础。

关键词：RAG；工作流编排；招生咨询；混合检索；智能问答系统

ABSTRACT

The challenge of university enrollment consultation lies not only in the number of queries, but also in the diversity of question types, the heterogeneity of information sources, and the strict requirement for factual correctness. In a single consultation session, a candidate may ask about admission policies, tuition, subject requirements, departmental websites, and registration procedures in sequence. Traditional FAQ systems are limited in coverage, while purely generative large language models are prone to hallucination in high-stakes admission scenarios. To address these issues, this thesis designs and implements an admission consulting system for Zhongyuan University of Technology based on Retrieval-Augmented Generation (RAG) and workflow orchestration.

The system adopts a front-end and back-end separated architecture. The front end is built with Vue 3, TypeScript, and Vite, and supports multi-session interaction, mode switching, and streaming responses through Server-Sent Events. The back end uses FastAPI as the unified gateway and organizes query rewriting, structured routing, hybrid retrieval, citation checking, answer generation, memory access, skill execution, and graceful degradation in a workflow manner. At the knowledge layer, unstructured enrollment documents are parsed and indexed with FAISS and BM25, while structured admission facts such as major catalogs, score lines, and policy tables are stored in MySQL to support precise field-level lookup.

Instead of relying on a single retrieval strategy, the system employs a hybrid pipeline composed of dense retrieval, sparse retrieval, exact match, reciprocal rank fusion, reranking, and quality gating. For complex consultation tasks, workflow orchestration is used to split the problem into traceable execution steps. To improve trustworthiness, the system further integrates citation validation, conflict resolution, input/output safety checks, health monitoring, isolation execution, and fallback mechanisms.

Experimental analysis is conducted based on existing project reports. In the dedicated

enrollment RAG evaluation with 70 cases, tool routing accuracy reaches 100%, while the overall pass rate is 55.71%, with Recall@5, MRR@5, and nDCG@5 of 0.6976, 0.9857, and 0.8078 respectively. These results indicate that the system performs well in routing and evidence recall, but still has weaknesses in answer faithfulness for college profile and guide-style questions. In sampled answer-quality evaluation, both keyword coverage and citation hit rate reach 100%, and no hallucination is observed. Fault-injection gateway replay shows that all five core cases can still return usable responses with a P95 latency of 13.98 ms. However, concurrency stress tests reveal that the current prototype suffers from high end-to-end latency, which means it is better suited to controlled demonstrations than to direct high-concurrency production deployment.

In conclusion, the thesis delivers a practical admission consultation prototype and verifies the feasibility of combining structured query, hybrid retrieval, workflow orchestration, and reliability control in a university-specific setting. The system still requires improvement in retrieval reranking, persistent memory, and deployment convergence, but it has already established a clear engineering foundation for trustworthy vertical question answering.

Keywords: RAG; Workflow Orchestration; Enrollment Consultation; Hybrid Retrieval; Intelligent Question Answering System

目 次

摘要 I
第一章 绪论 1
1.1 研究背景 1
1.2 国内外研究现状 2
1.2.1 检索增强生成研究现状 2
1.2.2 代理式推理与工作流编排研究现状 3
1.2.3 高校垂直问答系统研究现状 3
1.3 研究内容与目标 4
1.4 论文结构 4
第二章 相关技术与理论基础 6
2.1 大语言模型与检索增强生成 6
2.2 混合检索相关技术 6
2.2.1 稠密检索与向量索引 6
2.2.2 稀疏检索与 BM25 7
2.2.3 精确匹配、融合与重排 7
2.3 工作流编排与代理式问答 7
2.4 系统开发相关技术 8
第三章 需求分析与系统总体设计 9
3.1 业务场景分析 9
3.2 功能需求分析 9
3.3 非功能需求分析 10
3.4 系统总体架构设计 10
3.5 数据源与知识组织设计 11
第四章 系统关键模块设计与实现 13
4.1 招生知识库构建 13
4.2 混合检索增强模块设计 13
4.3 结构化招生查询模块设计 14
4.4 专家模式与工作流编排设计 15
4.5 会话记忆与技能机制设计 15
4.6 可靠性与安全控制设计 16
第五章 系统测试与结果分析 17
5.1 测试环境与方案设计 17
5.2 功能测试结果 18

5.3 招生 RAG 专项评测分析 18
5.4 检索消融实验分析 20
5.5 性能与容错测试分析 21
5.6 结果讨论 22
第六章 结论与展望 23
6.1 研究结论 23
6.2 不足与后续工作 23
参考文献 25
附录 28
附录 A 系统主要接口 28
附录 B 主要数据源与脚本 28
致谢 29

1 第一章 绪论

1.1 研究背景

高校招生工作近几年明显呈现出两个变化。其一，咨询入口越来越线上化，问题不再只来自招生现场和热线电话，而是同时分布在官网、公众号、咨询平台和临时活动页面上；其二，咨询内容越来越细碎，除了章程、计划、专业介绍等常规问题，考生和家长还会频繁追问某个省份、某个专业、某个年份的具体字段信息。对招生部门而言，这意味着简单的信息发布已经不够，必须提供更及时、更一致，也更容易被普通用户理解的答复服务。

传统 FAQ 系统在这一场景下有明显短板。它能覆盖高频固定问法，但很难处理自然语言表达差异，更难把分散在章程、学院介绍、专业目录、历年分数线和报到流程中的信息整合起来。纯人工咨询虽然更稳，但响应速度受限，且不同老师对同一问题的表述可能不一致。随着大语言模型在问答领域表现出较强的语言理解和表达能力，利用模型构建招生咨询系统看起来是一条自然的路线，但真落到业务上就会发现，模型“会说”不等于“能说准”。

招生咨询属于高谨慎场景。学费、选科要求、录取批次、报到时间、联系方式这些信息看起来都是小字段，一旦答错，对考生决策的影响却很直接。也正因为如此，招生问答系统的核心目标不能只停留在“回答自然”，还必须满足三条底线：第一，答案尽量来自可核验的学校资料；第二，涉及精确字段时尽量走结构化查询而不是语言猜测；第三，当证据不足时，系统宁可保守也不能装懂。

RAG 技术正是在这种背景下进入高校垂直场景的。它通过在生成前检索外部资料，让模型基于局部证据回答问题，从而缓解知识静态化带来的幻觉问题 [1, 2]。但从项目实现角度看，单纯“向量检索 + 大模型生成”仍然不够，因为招生问题里既有语义问答，也有数值字段、时间敏感信息和多步骤咨询。若系统只会统一走一条链路，很多该查库的问题反而会被错误地交给生成模型处理。

因此，本文把课题落点放在“RAG 与工作流的结合”上。这里的工作流，不是为了把系统包装得更花哨，而是为了把不同类型的问题分流到合适的能力模块：字段类问题优先走结构化查询，解释类问题走混合检索，复杂问题通过步骤化执行组织记忆、检索、工具与答案整合。在已有项目基础上，本文尝试回答一个更实在的问题：在高校招生这个高谨慎、强时效、资料格式又很杂的场景里，能不能做出一套答得相对稳、证据说得清、出了故障还能保底的智能咨询系统。

1.2 国内外研究现状

1.2.1 检索增强生成研究现状

RAG 已经成为大语言模型应用中最常见的知识增强路线之一。Lewis 等提出的基础框架把外部检索与文本生成结合起来，使模型在回答时不必完全依赖参数记忆，而是能够从外部资料中临时获取证据 [1]。后续研究不断扩大这一路线的边界，Gao 等从索引构建、检索策略、生成控制和评价方法等角度对大模型时代的 RAG 做了系统梳理，指出真正决定效果的往往不是单个模型，而是数据组织、召回策略和证据使用方式 [2]。

在工程实践中，纯向量检索并不能覆盖所有问题。对于金额、专业代码、年份和联系方式这类硬约束字段，BM25 等稀疏检索方法依然有很高价值 [3]；当系统需要同时参考多个召回通道时，RRF 之类的融合方法又能够在不引入复杂学习成本的前提下提升结果稳定性 [4]。Johnson 等提出的 FAISS 为本地向量索引提供了高效实现，使中小规模垂直语料库的相似检索部署门槛大幅降低 [5]。这些研究给本文的技术选择提供了直接依据。

不过，RAG 的难点并不只在“能不能找到资料”，还在“拿到资料以后怎么用”。如果证据排序不稳，或者模型没有按证据作答，系统依然可能在最终答案阶段出错。也就是说，检索增强并不会自动带来答案忠实性，证据质量控制和答案约束仍然需要在系统层面额外设计。

1.2.2 代理式推理与工作流编排研究现状

近两年，研究关注点逐步从“给模型更多文档”转向“让模型在任务中更主动地决定何时检索、何时推理、何时调用工具”。ReAct 将推理和行动交替组织，使模型能在任务执行中使用外部环境信息修正自身判断 [6]。Search-o1 则进一步把主动检索和长链路推理结合起来，探索在复杂问题中如何根据中间不确定点触发搜索，再把检索结果继续纳入后续推理过程 [7]。相关综述也显示，Agentic RAG 的发展方向已经不再局限于召回优化，而是开始强调任务规划、证据利用、执行轨迹可审计和多轮反馈闭环 [8, 9]。

这些研究对高校招生咨询尤其有启发。招生问题并不是一个固定模版任务，它常常同时包含“信息检索”“字段核对”“流程解释”和“上下文追问”几种需求。把它们都硬塞进单轮问答里，系统表现通常不会太稳定。工作流编排的价值，恰恰在于把任务拆成更适合执行和控制的步骤，让系统在每一步都能决定：接下来是检索、查库、重写问题，还是干脆给出保守说明。

1.2.3 高校垂直问答系统研究现状

高校问答系统并不是新事物，早期很多系统采用规则模板、关键词匹配或 FAQ 检索，能解决一部分固定问题，但很难应对跨文档整合和自然语言追问。随着大模型能力增强，越来越多场景开始尝试基于 LLM 的咨询服务。但对高校业务而言，能否引入大模型只是第一步，更关键的是如何把校内资料真正组织成可追溯的知识底座。

高校招生场景有几个典型特点。第一，资料来源多而杂，既有 Word、PDF、Excel，也有网页、Markdown 和人工整理文本。第二，字段表达不统一，同一个概念可能在不同资料里出现不同写法。第三，时效性强，去年和今年的政策不能混答。第四，部分问题本质上属于数据库查询，一旦交给生成式模型自由发挥，风险反而更高。现有很多通用问答系统对这类现实细节处理并不充分。

因此，围绕高校招生咨询做系统设计，真正值得研究的不是“大模型能不能回

答”，而是如何把结构化与非结构化知识结合起来、如何让系统在复杂问题中按步骤调用不同能力、以及如何用工程化手段尽量降低错误输出。本文的工作正是在这一缺口上展开。

1.3 研究内容与目标

本文研究对象是“基于 RAG 与工作流的中原工学院招生咨询系统”。与单纯做算法验证不同，本文更加关注真实业务资料、真实系统结构和真实评测结果，希望在不夸大能力边界的前提下，给出一套可复现、可解释的实现方案。

结合现有项目代码、文档与评测材料，本文主要完成了以下几项工作：

1. 设计并实现招生知识底座，完成对章程、学院介绍、招生专业详情、录取分数线和问答资料的清洗、解析、切块、索引和结构化入库。
2. 设计混合检索链路，将稠密检索、BM25 检索、精确匹配、RRF 融合、重排和质量闸门串联起来，支撑解释类问题与证据型回答。
3. 设计工作流式问答编排机制，将查询改写、结构化工具、记忆、技能和答案整合组织为可追踪步骤，用于处理复杂咨询任务。
4. 构建前后端完整原型，并基于现有脚本完成功能、检索、问答质量、性能和容错分析。

围绕这些工作，本文希望达成四个目标：第一，在招生资料异构且问题表达自然的条件下，提高系统的字段准确性和证据利用能力；第二，在复杂问题场景中，通过工作流编排提高系统处理过程的清晰度和可控性；第三，在依赖服务异常时仍尽量维持主链路可用；第四，为后续高校垂直问答系统建设提供具有工程可操作性的参考。

1.4 论文结构

全文共分为六章。

1. 第一章为绪论，主要介绍研究背景、国内外研究现状、研究内容与论文结构。

2. 第二章介绍系统设计所依赖的相关技术，包括大语言模型、RAG、混合检索、工作流编排以及系统开发技术基础。
3. 第三章从业务需求出发，对系统总体架构、模块划分和知识组织方式进行分析与设计。
4. 第四章详细说明系统关键模块的实现，包括知识库构建、混合检索、结构化查询、专家模式、记忆与可靠性设计。
5. 第五章结合现有测试和评测报告，对系统的功能效果、检索表现、消融结果、性能瓶颈和容错能力进行分析。
6. 第六章总结全文工作，指出系统当前不足，并给出下一步优化方向。

2　第二章　相关技术与理论基础

2.1　大语言模型与检索增强生成

大语言模型的优势在于语义理解和自然语言生成能力强，尤其适合把来源不同、表达方式不同的资料组织成连贯回答。但对于高校招生这种高风险问答场景，单靠模型参数中“记住的知识”是远远不够的。因为模型训练数据既不一定覆盖学校最新资料，也无法保证在回答时自动区分出哪个年份、哪个来源才是当前有效依据。

RAG 的基本思想，就是在生成前先从外部知识源中检索与问题相关的材料，再把这些材料和问题一起交给模型处理 [1]。相比纯生成方式，它最大的优点是让模型有机会“看见”当前语料库中的真实文本，从而提升回答的事实性和可解释性。Gao 等的综述指出，RAG 系统的效果通常受五类因素影响：数据组织方式、切块粒度、召回策略、证据排序方式以及答案生成约束 [2]。这说明 RAG 并不是一个单点算法，而更像一条完整的工程链路。

在本文的项目场景中，RAG 的价值主要体现在两个方面。第一，招生章程、报考指南、学院介绍等资料都具有较强文本性，适合通过切块与检索方式参与问答。第二，很多问题虽然最终需要自然语言表达，但其答案依据必须来自具体文本片段，例如某条政策说明、某个学院简介或某段报到须知。RAG 使系统能够在给出结论时附带来源，至少在机制上为“有据可查”提供了基础。

2.2　混合检索相关技术

2.2.1　稠密检索与向量索引

稠密检索通过把问题和文档映射到同一向量空间中来计算相似度，优点是能够处理词面差异较大的语义匹配问题。对于“计算机类专业有哪些发展方向”“人工智能学院主要培养什么能力”这类描述型问题，稠密检索通常比纯关键词匹配更稳。项目中使用 FAISS 作为本地向量索引，其优点是部署简单、检索速度快、适合中小规模

垂直知识库 [5]。

2.2.2 稀疏检索与 BM25

虽然向量检索对语义问题很有效，但在金额、年份、专业代码和专有名词等强约束场景中，它并不总是最优。BM25 这类稀疏检索模型对关键词与词频变化更敏感，往往能够在字段定位上提供更直接的支持 [3]。例如用户问“软件工程学费多少”“2025 年河南物理类最低分多少”，这些问题就更适合让稀疏检索参与召回。

2.2.3 精确匹配、融合与重排

招生资料里还有一类问题甚至不适合只靠相似检索处理，例如学院网址、联系电话、专业代码等短字段。对此，项目里额外设计了精确匹配通道，用来对疑似硬字段问题做规则化补强。多路召回结果再通过 RRF 融合到一个统一排序空间中，避免完全依赖单一路径 [4]。之后系统还会调用重排器做二次排序，理论上可以进一步提升候选证据与问题的相关性。

不过，重排并不天然带来收益。后文第五章的消融结果表明，在当前实现与当前数据条件下，查询改写和重排并没有稳定提升评测指标，反而带来额外时延。这也说明，检索系统的每一层设计都必须接受数据验证，不能仅凭理论印象判断效果。

2.3 工作流编排与代理式问答

工作流编排的核心价值，是把一个复杂问题拆成更可控的若干步骤。ReAct 的研究已经表明，当模型能够在推理过程中调用外部行动时，复杂任务的表现会明显不同于“直接一步回答”[6]。在招生咨询场景里，工作流编排同样重要：用户一句“我这个分数能报什么专业，顺便告诉我学费和学院联系方式”，背后往往就包含检索、查库、筛选和组织表达多个环节。

项目中采用 LangGraph 组织工作流节点，其好处在于流程边界清晰，便于插入查询改写、子问题拆分、结果审查和故障降级等控制逻辑 [11]。这种图式编排与传统

链式调用相比，更适合在节点之间做状态传递和重试控制，也更方便后续扩展额外能力。

代理式问答并不意味着系统必须“全自动联网思考”。在本文语境下，它更准确的含义是：系统能够根据问题类型，主动决定调用哪类能力，并把中间过程组织成可追踪的执行步骤。只要这一点做到了，工作流就已经对系统的可解释性和可维护性产生了价值。

2.4 系统开发相关技术

从实现层面看，项目采用的是前后端分离架构。前端使用 Vue 3、TypeScript 和 Vite，重点在于快速构建对话界面、多会话状态和 SSE 消息消费能力；后端使用 FastAPI 提供统一 API 入口，并借助 Pydantic 与 `pydantic-settings` 管理接口契约和运行配置 [10, 11]。

在数据处理方面，项目使用 `python-docx`、`PyPDF2`、`openpyxl` 和 `xlrd` 处理不同格式资料，再通过 MySQL 维护结构化招生表。这样的技术选型没有追求复杂，而是直接围绕业务资料形式展开，符合课程项目中“能稳定落地比堆技术名词更重要”的原则。

3　第三章　需求分析与系统总体设计

3.1　业务场景分析

本系统对应的是中原工学院招生咨询业务，目标用户主要包括考生、家长和招生工作人员。结合现有资料和项目实现，可以把咨询场景概括为三类。

第一类是政策说明类问题，例如“招生章程里对体检有哪些要求”“学校资助政策包括哪些内容”“新生报到需要注意什么”。这类问题通常需要系统从非结构化资料中抽取多个片段，再整合成可读回答。

第二类是字段精确类问题，例如“某专业学费是多少”“某专业代码是什么”“某省某批次最低录取分数线是多少”。这类问题虽然表面上也是自然语言输入，但本质更接近数据库查询，回答正确与否取决于结构化数据是否完整和路由是否准确。

第三类是复合型问题，例如“我的分数能报什么专业，还想了解学费和学院情况”“如果我报人工智能专业，报到前需要准备什么材料”。这类问题往往同时涉及字段筛选、资料检索和结果组织，更适合通过步骤化工作流处理。

3.2　功能需求分析

围绕上述业务场景，系统需要满足以下功能要求。

1. 普通对话与 RAG 问答：支持用户直接提问，并在需要时返回带来源说明的答案。
2. 结构化招生查询：对学费、学制、分数线、专业代码、选科要求等问题给出准确字段结果。
3. 专家模式编排：对复杂问题支持子问题拆分、步骤执行与结果整合。
4. 多会话与上下文承接：支持用户在同一会话中持续追问。
5. 运维与管理接口：支持健康检查、指标查看、索引重建和上下文压缩。

这些需求看似分散，实际都围绕一个核心目标：让系统在保证可用性的前提下，

把不同类型的问题导向合适的处理路径，而不是把所有问题都交给同一个模型调用。

3.3 非功能需求分析

招生咨询系统除了功能完整，还要满足一组更难的非功能指标。

准确性方面，系统需要尽量减少字段类错误和年份混答。可追溯性方面，系统需要在回答中保留来源上下文或引用信息，以便用户和管理者核验。可靠性方面，系统要在部分依赖失败时保持核心链路可用，而不是整轮崩掉。可扩展性方面，后续应能继续接入新的知识源、新的结构化表和新的工作流节点。可维护性方面，模块边界要清楚，便于在毕业设计后续阶段继续演示、修复和扩展。

这些要求决定了系统不能只是一组模型调用脚本，而必须具备比较完整的工程结构。

3.4 系统总体架构设计

结合代码仓库结构，系统总体上采用“表现层 + 接入层 + 编排层 + 能力层 + 数据层”的五层设计 [11]。表现层由前端界面构成，负责会话管理、模式切换与流式展示；接入层由 FastAPI 接口组成，负责统一接收请求和返回结果；编排层负责问题路由、执行计划和降级控制；能力层负责 RAG、结构化查询、记忆、技能和生成；数据层负责原始资料、向量索引和关系型表。

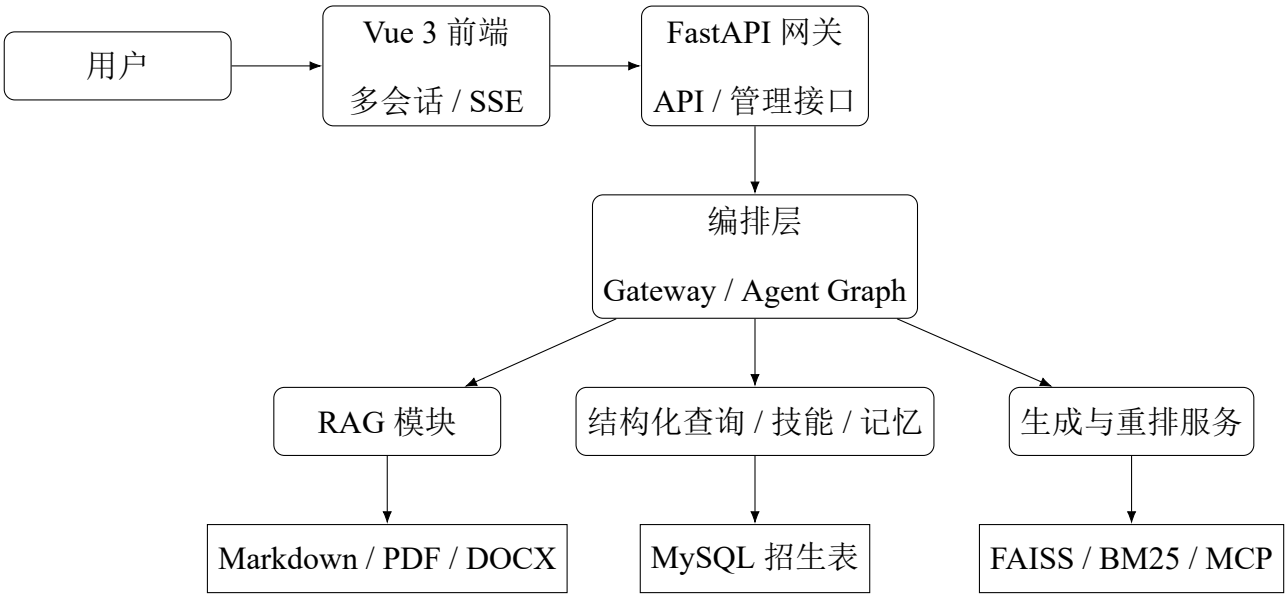


图 3-1 系统总体架构

这种分层结构的意义在于职责明确。前端不直接承担业务决策，后端入口不直接拼接全部逻辑，RAG 不负责记忆，结构化查询也不替代生成模块。每个模块只做自己该做的事，复杂流程则由编排层组织，这样更有利于后续定位问题和控制风险。

3.5 数据源与知识组织设计

从项目现有资料统计看，系统当前可用原始文档共 78 份，其中与招生主链路直接相关并进入索引清单的源文件为 42 份；结构化解析目录中生成了 24 份 JSON 结果文件；后端自动化测试文件 23 个，前端测试文件 6 个。这些数据说明项目并非空壳原型，而是已经形成了可持续维护的资料和验证基础。

表 3-1 系统数据与验证资产概况

类别	数量	说明
原始资料文件	78	文档、表格、网页导出等原始知识源
索引清单纳入源文件	42	进入 FAISS/BM25 主链路的招生资料
结构化解析 JSON	24	解析后的学院介绍与章程等结果
后端测试文件	23	API、RAG、Agent、门禁、并发等测试
前端测试文件	6	组件、API 服务与请求构造测试

知识组织上，系统没有把所有资料一股脑混在一起，而是分成两套互补底座。第一套是非结构化知识库，主要承载招生章程、报考指南、学院介绍和说明性材料，适合做语义检索与证据型回答。第二套是结构化招生表，主要承载专业目录、学费、学制、录取分数线、政策附表等字段信息，适合做精确查找。两套底座在接口层被统一包装，这使系统可以根据问题类型灵活选择主路径，而不必强行统一处理。

4 第四章 系统关键模块设计与实现

4.1 招生知识库构建

知识库构建是系统能否回答招生问题的基础。项目中的数据处理脚本会读取 PDF、DOCX、XLS 和 XLSX 等多种格式资料，将原文内容、标题、来源、发布日期和主题等元数据整理出来，再分别进入结构化解析或非结构化切块流程 [11, 10]。对于结构化程度较高的表格型资料，系统直接抽取字段并写入数据库；对于说明性文档，则切分为适合检索的文本块。

从当前实现看，文档切块并不是简单的等长分段，而是结合摘要层和小块层组织内容。这样做的好处是：摘要层能帮助系统先定位主题范围，小块层再提供更精确的证据片段，二者配合后，系统在长文档中的检索体验会明显好于单层切块。与此同时，索引目录中还保存了 `manifest.json`，用于记录文件列表、切块参数和语料指纹，避免系统误加载陈旧索引。

4.2 混合检索增强模块设计

混合检索模块是本文系统的核心之一。它不是把一个问题直接送去向量搜索，而是按步骤进行：先做查询标准化和必要的改写，再分别执行稠密检索、BM25 检索与精确匹配，之后对候选结果做去重、融合和重排，最后经过质量闸门与引用校验，再决定是否将证据交给生成模块。

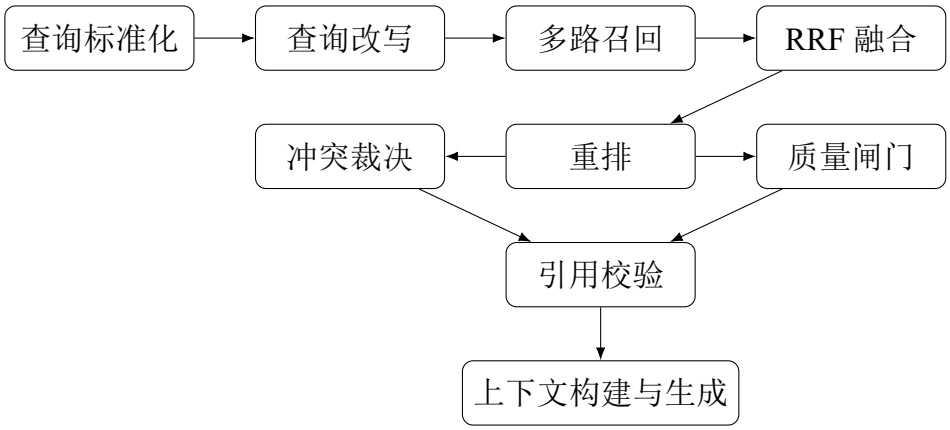


图 4-1 混合检索增强链路

这种设计的好处在于，每一步都承担了明确职责。稠密检索负责语义召回，BM25 负责关键词和数值敏感问题，精确匹配负责硬字段补强，RRF 负责融合，重排负责局部精修，质量闸门负责过滤低覆盖、冲突或过期证据，而引用校验则对最终答案进行来源约束。相比“检几段就回答”的简单方案，这条链路更适合高校招生这类对事实正确性敏感的场景。

不过，系统当前的消融报告也说明，这条链路仍需要继续调参。现有 48 例消融结果表明，`exact_only` 在 MRR 上达到 0.9792，说明字段类问题非常适合精确匹配；`rrf_hybrid_small2big_off` 的 Recall@5 达到 0.8333，说明当前小块主导时召回更充分；而查询改写与重排在现阶段反而拉低了总体指标，说明这两部分的策略还不够成熟，后续不能想当然地继续加深复杂度。

4.3 结构化招生查询模块设计

结构化招生查询模块的目标很明确：凡是可以通过字段直接回答的问题，尽量不要让模型自由生成。项目中通过路由器识别问题类型，把分数线、位次问题送入 `score_lines`，把学费、学制、专业代码和选科要求送入 `major_catalog`，把政策附表类问题送入 `policy_tables`[11]。

这种分流方式在第五章的评测中表现得非常明显。专业目录类样本共 30 条，全部通过，平均得分 0.9799；而学院介绍类样本通过率只有 22.22%。这说明结构化链

路在字段问题上的稳定性显著优于纯文本问答，也证明了把强约束问题从生成链路中剥离出来是一个正确方向。

结构化模块还实现了本地缓存回退。当数据库不可用或查询为空时，系统可以从缓存文件中做补救检索，再统一包装成结果返回。这种兜底虽然比主链路弱，但至少保证了系统不会因为单一存储层异常而完全失效。

4.4 专家模式与工作流编排设计

专家模式是系统对复杂问题的处理入口。它并不是为了把系统包装成“万能智能体”，而是为了解决单轮问答无法妥善处理的复合型咨询任务。根据中期报告和现有代码，专家模式已经具备“加载会话记忆—查询前处理—子问题拆分—生成执行计划—依次调用本地 RAG、结构化查询、技能或 MCP 工具—审查结果—必要时重试或降级—汇总答案—异步写回记忆”的完整链路 [11]。

这种设计的直接价值在于两点。其一，复杂问题能够被拆成若干更小的执行单元，降低单次回答对模型长链推理的依赖。其二，步骤化执行便于插入状态记录、失败审查和策略切换。项目中还区分了 `speed` 与 `quality` 两种策略，前者更偏响应速度，后者允许更多次重试和一次重规划。

需要如实说明的是，现阶段专家模式仍存在稳定性不足的问题。中期报告已经指出，多次测试中专家模式还不能很好地针对复杂问题自主探索新方案，有时会在结果不足时直接把问题抛回给用户。这说明当前工作流框架已经搭起来，但复杂任务规划能力还没有完全成熟。论文把这一点写清楚，比硬吹成“已实现全自动智能体”要稳得多。

4.5 会话记忆与技能机制设计

为支持多轮追问，系统实现了短期记忆、长期记忆和特殊记忆三类结构。短期记忆保存当前会话近期上下文，长期记忆以滚动摘要方式沉淀持续话题，特殊记忆则用于记录回答风格偏好等软约束。这样的设计虽然不复杂，但对“上一轮说的是河南

考生，这一轮继续问人工智能专业行不行”这种追问场景很有帮助。

技能机制则更偏扩展层。项目支持技能保存、版本去重和最优版本激活，使某些高频专题流程后续有可能被沉淀为固定技能。例如资助政策解读、报到流程说明、志愿填报辅助等，都可以沿着这条思路继续扩展。不过从当前实现看，技能依然更接近“规则化封装能力”，距离真正稳定的自治式任务学习还有明显距离。

4.6 可靠性与安全控制设计

招生咨询系统与普通聊天系统最大的区别之一，就是错误代价更高。因此，项目在可靠性和安全控制上花了不少力气。首先，网关引入了隔离执行器，对 RAG、记忆、技能等依赖做失败计数和熔断控制，避免某个能力连续异常时拖垮整轮回答。其次，系统通过引用校验和质量闸门控制证据数量、得分、冲突和时效性，当条件不满足时，优先返回保守说明而不是给出确定结论。

安全方面，系统对输入和输出都做了基本审查。对于“输出系统提示词”“忽略全部规则”等越权指令，网关会拦截；对于生成结果中可能出现的内部提示、疑似密钥或不应外露的内容，也会做过滤处理。虽然这套规则还谈不上完备，但对毕业设计阶段的招生咨询原型来说，已经体现了较清晰的安全边界意识。

5 第五章 系统测试与结果分析

5.1 测试环境与方案设计

项目的测试工作分为四层：第一层是前后端自动化测试，用于验证接口、组件和关键模块逻辑；第二层是问答与网关评测，用于检查核心样例能否返回有效结果；第三层是招生 RAG 专项评测，用于分析不同类型问题下的召回、路由和答案效果；第四层是并发压测与故障注入，用于检查主链路时延和降级表现。

表 5-1 测试环境与验证资产

项目	配置/数量	说明
后端框架	FastAPI + Python	统一网关与服务层
前端框架	Vue 3 + TypeScript + Vite	多会话与 SSE 展示
结构化存储	MySQL	专业目录、分数线、政策附表
向量与检索	FAISS + BM25 + Exact Match	混合检索主链路
后端测试文件	23 个	API、RAG、Agent、门禁等
前端测试文件	6 个	组件、API 服务、请求构造
RAG 专项评测样本	70 条	多类别招生问答样本
消融评测样本	48 条	检索模块对比实验样本

从测试材料规模看，这个项目已经不是单纯手点页面的演示性质原型，而是基本形成了“代码测试 + 评测脚本 + 结果报告”的闭环。虽然它距离完整生产级验证还有差距，但对于本科毕业设计而言，已经足够支撑系统分析和效果讨论。

5.2 功能测试结果

根据现有网关回放与自动化测试结果，系统已经能够稳定提供普通聊天、流式聊天、健康检查、索引重建、记忆压缩、特性查询和已保存技能列表等接口 [10, 12]。前端侧已经具备多会话切换、模式切换、步骤事件展示和右侧运维面板等能力，中期演示和联调所需的核心交互链路基本完整。

表 5-2 核心功能验证情况

功能项	验证结果	说明
普通聊天接口	通过	可返回同步问答结果
SSE 流式聊天	通过	支持 message/done 事件
模式切换	通过	支持 chat、rag、agent 三种入口
结构化招生查询	通过	专业目录类问题表现稳定
健康检查与指标接口	通过	支持依赖状态查看与后台管理
索引重建与上下文压缩	通过	已形成基本运维闭环

这部分测试结果说明，系统在“能不能用”这个问题上已经没有太大障碍，真正需要进一步讨论的是“在不同问题类型上答得怎么样”以及“负载上来后还能不能稳住”。

5.3 招生 RAG 专项评测分析

项目中的招生 RAG 专项评测覆盖 70 个样本。报告显示，整体工具路由准确率达到 100%，说明系统对于“该走结构化工具还是该走 RAG”的入口判断比较稳定；但总体通过率只有 55.71%，平均总分为 0.7443，说明系统在最终答案阶段仍有明显短板

[13]。

表 5-3 招生 RAG 专项评测总体结果

样本数	通过数	通过率	Recall@5	MRR@5	nDCG@5
70	39	55.71%	0.6976	0.9857	0.8078

从这组数据可以看出一个很有意思的现象：检索侧指标并不差，尤其 MRR@5 达到了 0.9857，说明正确证据往往已经出现在很靠前的位置；但答案层的通过率却没有同步上去。这意味着系统当前的主要矛盾并不完全在“没找到证据”，而更多在“找到以后没有老老实实按证据回答”。报告中的典型失败样例也支持这一判断，例如学院网址和联系电话问题里，证据明明给出了正确字段，但最终答案要么过度保守拒答，要么生成了与证据不一致的链接或号码。

表 5-4 不同问题类别的专项评测结果

类别	样本数	通过率	平均总分
专业目录	30	100%	0.9799
招生章程	2	50%	0.7208
报考指南	2	0	0.4685
学院介绍	36	22.22%	0.5645

类别分布进一步说明了系统当前的强弱项。专业目录问题全部通过，说明结构化查询链路有效；学院介绍问题表现最弱，说明纯文本场景下的证据利用和答案忠实性仍不稳定。对毕业设计来说，这样的结果并不丢人，反而说明系统分析有抓手：强项在哪，弱项在哪，都有数据支撑，不是拍脑袋下结论。

5.4 检索消融实验分析

为了进一步理解检索链路中各个组件的实际作用，项目对 48 个样本进行了消融实验，对比查询改写、重排、Small2Big 与单路检索变体的效果。结果如表 5-5 所示。

表 5-5 检索消融实验结果

方案	Recall@5	MRR@5	nDCG@5	平均时延/ms
rrf_hybrid_rewrite_off	0.3550	0.8167	0.5393	1822.86
rrf_hybrid_rewrite_on	0.2642	0.7917	0.4407	2183.65
rrf_hybrid_small2big_off	0.8333	0.6375	0.6862	1546.11
rrf_hybrid_small2big_on	0.3550	0.8167	0.5393	1643.46
rrf_hybrid_rerank_off	0.3550	0.8167	0.5393	1864.78
rrf_hybrid_rerank_on	0.2777	0.6208	0.3966	2548.38
bm25_only	0.1002	0.3646	0.1611	123.84
dense_only	0.1982	0.5042	0.2812	577.07
exact_only	0.3820	0.9792	0.6066	20.32

这组结果很有信息量。首先，`exact_only` 的 MRR 最高，说明在字段型样本上，精确匹配对“第一条就命中正确结果”非常有效。其次，`bm25_only` 与 `dense_only` 单独使用都不够理想，证明单路检索不足以覆盖当前招生语料。再次，查询改写和重排在当前实现里并没有带来正收益，反而拉低了指标并提高了时延，说明这两项能力的质量还没有达到上线级稳定状态。最后，`small2big_off` 的 Recall@5 反而更高，提示当前父子块组织和召回权重还需要继续调整。

对系统改进而言，这些结论非常重要，因为它直接告诉我们：问题不在于“组件不够多”，而在于“哪些组件已经有效，哪些组件只是增加了复杂度”。这类结果也提醒后续优化必须以数据为准，而不能凭印象继续叠模块。

5.5 性能与容错测试分析

性能测试方面，项目对 RAG 主链路做了并发压测。结果显示，系统在正确性上能扛住请求，但时延偏高，尤其在并发上升后增长明显 [15]。表 5-6 选取了三个代表性并发级别。

表 5-6 RAG 主链路代表性并发表现

并发数	错误率	P95 首字节/ms	P95 完整耗时/ms	结果
5	0	48773.56	56605.64	FAIL
30	0	76781.72	95837.11	FAIL
100	0	160244.26	218692.41	FAIL

这组数据非常直接地表明：当前系统虽然能在实验环境下完成任务，但距离高并发实用还有不小差距。问题倒也不神秘，主要就是检索、重排和生成整条链路都比较重，而很多优化还没有真正做完。把这个现实写出来，并不会削弱论文价值，反而说明系统分析没有回避问题。

与高时延形成对比的是容错回放结果。网关故障注入测试中，5 组核心样例全部返回有效响应，P95 延迟仅 13.98 ms；在 `rag` 与 `skill_exec` 故障注入下，系统也都能以 `degraded` 状态返回保守结果 [12, 15]。这说明项目当前的优势更偏“可保底”，而不是“高性能”。

表 5-7 故障注入与降级能力结果

场景	请求数	降级返回数	P95 首字节/ms	P95 完整耗时/ms
网关核心回放	5	4	13.98	24.97
注入故障：rag	8	8	87.08	15313.99
注入故障：skill_exec	8	8	5771.84	12353.36

5.6 结果讨论

综合第五章全部结果，可以把系统当前状态概括为三句话。第一，结构化查询链路是当前最稳的部分，尤其适合专业目录、分数线和学费类问题。第二，RAG 检索链路已经具备较好的召回能力，但答案忠实性仍不够稳定，特别是学院介绍和报考指南类问题。第三，系统的降级和保底机制做得比性能优化更成熟，这让它在课程原型阶段更适合“演示正确路径与失败处理方式”，而不是直接承诺高并发可用。

这种结果其实很符合一个真实毕业设计的演进阶段。项目已经把“能用、能测、能分析”的底子打出来了，但距离“足够快、足够稳、足够泛化”还有明显路要走。把这个边界讲清楚，反而比一味强调“全部达标”更有说服力。

6 第六章 结论与展望

6.1 研究结论

本文围绕中原工学院招生咨询业务，完成了一套基于 RAG 与工作流编排的智能咨询系统设计与实现。系统不是单纯把大模型接到页面上，而是结合结构化招生表、非结构化文档知识库、混合检索链路和工作流编排机制，形成了较完整的问答主链路。通过这一设计，系统能够分别处理解释类问题、字段类问题和部分复合型问题，并在回答中尽量保留来源依据和风险边界。

从系统实现来看，本文工作至少证明了三点。第一，针对高校招生场景，把结构化查询和 RAG 分开设计是必要的，字段问题走结构化链路能够显著提高稳定性。第二，工作流编排对复杂问题处理和故障控制都有实际价值，它让系统具备了按步骤调用不同能力的可能，而不是把所有任务都压给一次生成。第三，引用校验、健康检查、熔断和降级等工程措施虽然不“花哨”，但恰恰决定了这类高谨慎问答系统能不能真正落地。

从评测结果来看，系统当前在工具路由和证据召回方面表现较好，结构化问题效果稳定，抽样问答质量也较为理想；但在学院介绍和报考指南类问题上，答案忠实性仍需继续提升，说明最终答案生成阶段对证据的利用还不够扎实。并发压测则进一步说明，系统的可靠性设计已经有一定基础，但性能优化还没有完成。

6.2 不足与后续工作

当前系统仍存在以下不足。

1. 检索排序与答案忠实性之间还存在断层。正确证据很多时候已经被召回，但最终答案没有始终做到严格按证据作答。
2. 查询改写、重排和 Small2Big 等高级检索策略在当前数据条件下尚未稳定产生正收益，需要继续调参和缩小作用范围。

3. 专家模式具备流程框架，但复杂问题下的计划生成与自主探索能力仍不稳定。
4. 记忆仍以进程内实现为主，跨实例共享和长期持久化尚未落地。
5. 主链路在并发压力下时延偏高，当前更适合课程演示和受控环境，而不是高并发生产场景。

下一步工作可重点围绕以下方向展开：一是针对学院介绍、联系方式和网址等易出错问题，增强实体约束、字段抽取和答案模板约束；二是重新评估查询改写与重排的触发条件，只在确实有收益的场景中启用；三是补齐记忆持久化和部署层组件打通；四是继续扩充评测集，尤其增加跨年份、跨院系和长尾追问样本；五是在保证可靠性的前提下，再逐步优化端到端时延。

总的来说，本文并没有把系统包装成一个“已经完美”的招生智能体，而是给出了一套更接近真实工程状态的毕业设计成果：它已经能运行、能评测、能发现问题，也已经证明了这条技术路线是走得通的。后续只要围绕薄弱环节继续打磨，系统还有较大的提升空间。

参考文献

参考文献

[1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[EB/OL]. arXiv:2005.11401, 2020.

[2] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[EB/OL]. arXiv:2312.10997, 2023.

[3] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.

[4] Cormack G V, Clarke C L A, Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods[C]//Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2009: 758-759.

[5] Johnson J, Douze M, Jégou H. Billion-Scale Similarity Search with GPUs[EB/OL]. arXiv:1702.08734, 2017.

[6] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. arXiv:2210.03629, 2023.

[7] Li X, Dong G, Jin J, et al. Search-o1: Agentic Search-Enhanced Large Reasoning Models[EB/OL]. arXiv:2501.05366, 2025.

[8] Singh A, Ehtesham A, Kumar S, et al. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG[EB/OL]. 2025.

[9] Li Y, Zhang W, Yang Y, et al. Towards Agentic RAG with Deep Reasoning: A Survey of RAG-Reasoning Systems in LLMs[EB/OL]. 2025.

[10] Enrollment-Consultation-System 项目组. Enrollment-Consultation-System README[Z]. 2026.

[11] Enrollment-Consultation-System 项目组. 系统架构与技术栈分析、毕业设计正文素材、技术栈证据清单 [Z]. 2026.

[12] Enrollment-Consultation-System 项目组. 网关评测结果报告 [Z]. `backend/reports/eval_report.json`, 2026.

[13] Enrollment-Consultation-System 项目组. 中原工学院招生 RAG 专项评测报告 [Z]. backend/reports/rag_eval_report.json, 2026.

[14] Enrollment-Consultation-System 项目组. 问答质量评测报告 [Z]. reports/answer_eval_report.json, 2026.

[15] Enrollment-Consultation-System 项目组. 本地压测摘要报告 [Z]. backend/reports/perf/20260511-210125_local-rehearsal/perf_summary.json, 2026.

[16] 中原工学院本科毕业设计课题资料. 基于 RAG 与工作流的中原工学院招生咨询系统 [Z]. 2026.

附录

附录 A　系统主要接口

系统后端当前提供的主要接口包括：

- /api/chat 与 /api/chat/stream：普通问答与流式问答；
- /api/features 与 /api/mcp/tools：能力配置与工具清单；
- /api/skills/saved 与 /api/memory/compress：技能与记忆管理；
- /api/admin/metrics、/api/admin/reindex 与 /healthz：后台运维、索引维护与健康检查。

这些接口共同覆盖了普通问答、流式输出、功能查询、技能治理、运维观测和索引维护等主要需求。

附录 B　主要数据源与脚本

系统当前直接使用或经处理后使用的核心数据源包括中原工学院 2025 年普通本科招生章程、2025 报考指南、2025 年招生专业详情、学院介绍资料、历年录取分数线以及问答库。主要脚本包括：

- build_admissions_kb.py：结构化知识库构建；
- evaluate_gateway.py 与 evaluate_retrieval.py：网关与检索评测；
- ablate_retrieval.py：检索消融实验；
- perf_chat_stream.py：并发压测；
- gate_release.py：发布门禁判断。

致谢

毕业设计从系统原型搭建到论文写作，前后跨度较长，涉及代码实现、资料整理、测试验证和版式规范等多项工作。感谢指导教师车战斌老师在课题方向、研究边界和写作规范上的指导；也感谢项目中积累下来的测试脚本、评测报告和技术分析文档，它们为本文撰写提供了大量可以追溯的事实依据。通过本次毕业设计，我对“把一个能跑的功能做成一个能分析、能验证、能交付的系统”这件事，有了更具体也更扎实的理解。